

Building and Simulating Circuits

13, 15 October · Objective 4 · Quiz 2 this week

This week is craft: constructing circuits that do something, verifying them against theory, and learning which simulator to reach for. The habit to build is verifying on a statevector simulator *before* spending shots — on hardware or in a sampler — because a circuit that is wrong in theory will not become right with more samples.

Choosing a simulator

BACKEND	RETURNS	USE FOR	LIMIT
Statevector	exact amplitudes	verifying logic, debugging	~30 qubits (2 ⁿ memory)
AerSimulator	sampled counts	realistic shot statistics	~30 qubits
AerSimulator(method='matrix_product_state')	counts	low-entanglement circuits	100s of qubits
AerSimulator(noise_model=...)	counts	pre-hardware validation	~25 qubits
Real backend	counts	the actual answer	queue time, noise

The workflow that saves the most time: **Statevector first**. It gives you the exact state, so a logic error is immediately visible rather than hidden under sampling noise.

A worked circuit: GHZ state

The three-qubit generalization of a Bell state, $(|000\rangle + |111\rangle)/\sqrt{2}$:

```

from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector
from qiskit_aer import AerSimulator

def ghz(n):
    qc = QuantumCircuit(n)
    qc.h(0)                # seed the superposition on one qubit
    for q in range(n - 1):
        qc.cx(q, q + 1)    # propagate correlation down the chain
    return qc

qc = ghz(3)
print(Statevector(qc))
# [0.707+0j, 0, 0, 0, 0, 0, 0, 0.707+0j] -> only |000> and |111> have amplitude

```

The linear chain of CNOTs is the readable construction, not the fastest. On hardware with limited connectivity a **logarithmic-depth** version wins:

```

def ghz_log_depth(n):
    qc = QuantumCircuit(n)
    qc.h(0)
    step = 1
    while step < n:        # double the entangled block each round
        for q in range(0, min(step, n - step)):
            qc.cx(q, q + step)
        step *= 2
    return qc

```

Depth $O(\log n)$ instead of $O(n)$ — which matters because coherence time is fixed, so shallower circuits accumulate less decoherence. Depth, not gate count, is the quantity to minimize on NISQ hardware.

Verifying against theory

Never trust a circuit you have not checked. Three levels:

```

from qiskit.quantum_info import Statevector, Operator, state_fidelity
import numpy as np

# 1. State-level: does it produce the state you intended?
target = Statevector([1/np.sqrt(2), 0, 0, 0, 0, 0, 0, 1/np.sqrt(2)])
print(state_fidelity(Statevector(ghz(3)), target))    # 1.0

# 2. Operator-level: is the whole unitary what you think?
# Stronger than state checks – it verifies behavior on ALL inputs.
print(Operator(qc_a).equiv(Operator(qc_b)))          # True if equal up to global phase

# 3. Structural: what will actually run?
qc_t = transpile(qc, backend, optimization_level=3)
print(qc_t.depth(), qc_t.count_ops())

```

Level 2 is the one people skip. A circuit can produce the right output on $|000\rangle$ and be wrong everywhere else — which surfaces only when you use it as a subroutine.

Analogy boundary — statevector inspection as a debugger. Reading amplitudes mid-circuit feels like inspecting variables at a breakpoint, and for *simulation* the analogy is exact. It fails completely on hardware: there is no way to observe a real quantum state without measuring and destroying it. Statevector debugging is available only in the classical simulation of your circuit — which means it stops working precisely when your circuit gets big enough to be interesting. Design for verifiability at small n , then scale.

Measurement and shot statistics

Sampling error falls as $1/\sqrt{N}$. To resolve a probability to $\pm\epsilon$ you need $N \approx 1/\epsilon^2$ shots:

```

±0.10 →    100 shots
±0.01 → 10,000 shots
±0.001 → 1,000,000 shots

```

This is a hard economic constraint. Variational algorithms requiring many expectation values to high precision can spend hours of device time on a single optimization step, and it is the reason "shots" appears in every hardware cost model.

```

sim = AerSimulator()
qc_m = ghz(3); qc_m.measure_all()
for shots in (100, 10_000):
    c = sim.run(transpile(qc_m, sim), shots=shots).result().get_counts()
    p = c.get('000', 0) / shots
    print(f"{shots:>6}: P(000) = {p:.4f} (expect 0.5000)")

```

Quiz 2 review

Covering mathematical foundations, gates, and circuits.

- **Bloch half-angles.** $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$. Orthogonal $\Rightarrow$ antipodal.
- **Gate identities.** $HZH = X$, $HXH = Z$, $S^2 = Z$, $T^4 = Z$, $T^8 = I$. All Paulis square to I.
- **Separability.** $ad - bc = 0 \iff$ product state.
- **Little-endian.** Qiskit prints qubit 0 rightmost.
- **Clifford vs. T.** H, S, CNOT are classically simulable; T supplies universality.
- **Hermitian = observable, unitary = evolution.** The Paulis are both.
- **Depth vs. count.** Depth is the coherence-limited quantity.

Key takeaways

- Verify with `Statevector` before spending shots; exact amplitudes expose logic errors that sampling noise hides.
- `Operator.equiv` checks behavior on all inputs — strictly stronger than checking one output state.
- Circuit **depth** governs decoherence exposure; log-depth constructions beat linear ones on hardware.
- Shot count scales as $1/\epsilon^2$. Precision is purchased at quadratic cost.
- Statevector debugging exists only in simulation, and disappears exactly when circuits become large enough to matter.

Self-check

Q1 (Objective 4) — Two GHZ constructions produce identical statevectors on $|000\rangle$ but differ in depth. Explain why `state_fidelity` = 1.0 is insufficient grounds to call them interchangeable, and give a test that would distinguish them.

Q2 (Objective 4) — You need $P(\text{outcome})$ to $\pm 0.5\%$. How many shots, and what does this imply for a variational algorithm requiring 200 such estimates per optimization step across 50 steps?

Check your answers

A1 — `state_fidelity` on one input checks a **single column** of the unitary. Two circuits agreeing on $|000\rangle$ may disagree on every other basis state, and the difference surfaces the moment either is used as a controlled subroutine or applied to a superposed input — which is exactly how subroutines are used in Grover and Shor.

Two tests, in increasing strength:

```
# Operator equivalence: compares the full  $2^n \times 2^n$  unitary, all inputs at once.
Operator(qc_a).equiv(Operator(qc_b))      # up to global phase

# If that is too large, sample random inputs — cheaper, probabilistic.
for _ in range(20):
    psi = random_statevector(2**n)
    assert state_fidelity(psi.evolve(qc_a), psi.evolve(qc_b)) > 1 - 1e-9
```

Beyond correctness, they are not interchangeable **operationally** even if the unitaries match exactly. The log-depth version has shorter depth, so it accumulates less decoherence; the linear version may transpile better on a linear-topology device by requiring no SWAPs. Unitary equivalence is a statement about the mathematical map, and says nothing about fidelity on hardware. Both facts matter, and they are checked by different means.

A2 — For $\epsilon = 0.005$, $N \approx 1/\epsilon^2 = 40,000$ shots per estimate.

Scaling up:

```
40,000 shots × 200 estimates × 50 steps = 400,000,000 shots
```

At a realistic 1–10 kHz effective sampling rate including reset and readout, that is **11 to 110 hours** of device time — for one optimization run, assuming no queue and no re-runs. This is the central practical obstacle to variational quantum algorithms, and it is usually understated in papers that report only circuit depth.

The mitigations, all partial: reduce precision in early optimization steps and tighten as you converge; group commuting observables so one set of shots yields several expectation values; use shot-frugal optimizers (SPSA, Rosalin) that tolerate noisy gradients; and reduce the observable count through symmetry.

The lesson worth carrying: quantum advantage claims must account for shot overhead. An algorithm with an asymptotic speedup that needs 10^8 samples per step may lose to a classical method that needs none — and comparisons that omit shot cost are not comparisons.